

Informe del proyecto final



Fundamentos De Ingeniería De Software

Sebastian Alejandro Gaibor

Laura Valentina Ladino Rivera

Richard Arley Navas Peñaralta

Dana Lizeth Trujillo Rivera

Luis Moreno Sandoval

13/11/2025

1. Métricas de Calidad.

a. Métricas Implementadas:

- i. La complejidad ciclomática mide la cantidad de caminos lógicos posibles en el código. Cuanto menor sea el valor, más sencillo y fácil de probar es el programa.
- ii. El índice de mantenibilidad estima qué tan fácil es comprender, modificar y extender el código (se basa en la cantidad de líneas de comentarios y en la complejidad ciclomática previamente calculada). Valores altos indican mejor calidad estructural y legibilidad.
- iii. Captura de pantalla de algunos resultados de las métricas:

```
Archivo analizado: Usuario.java
Complejidad ciclomática: 1  Baja (código simple y fácil de mantener)
Índice de mantenibilidad: 302,98  Excelente (muy fácil de mantener)
-----
Archivo analizado: UsuarioController.java
Complejidad ciclomática: 1  Baja (código simple y fácil de mantener)
Índice de mantenibilidad: 251,92  Excelente (muy fácil de mantener)
-----
Archivo analizado: UsuarioRepository.java
Complejidad ciclomática: 1  Baja (código simple y fácil de mantener)
Índice de mantenibilidad: 165,10  Buena (fácil de mantener con pocas mejoras)
-----
Archivo analizado: UsuarioService.java
Complejidad ciclomática: 2  Baja (código simple y fácil de mantener)
Índice de mantenibilidad: 158,16  Buena (fácil de mantener con pocas mejoras)
-----
```

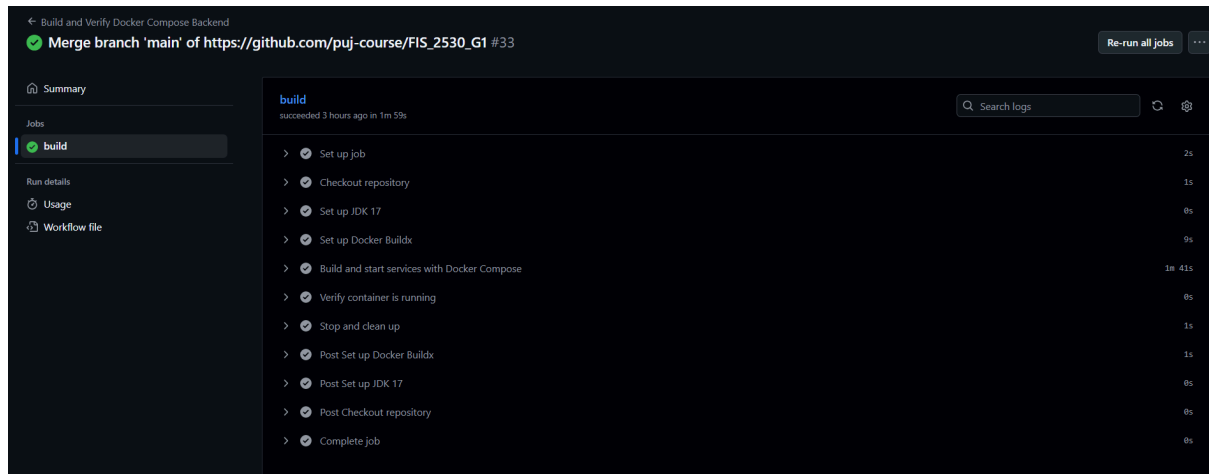
b. Resumen general de las métricas encontradas en SonarQube

Métrica	Valor Reportado	Estado	Observación
Líneas de Código (LOC)	1.2k (Java, XML)	N/A	Total de código analizado.
Cobertura (Coverage)	11.2%	Crítico	Bajo nivel de cobertura de pruebas unitarias. Se recomienda aumentar la cantidad de tests.
Deuda Técnica (Maintainability)	173	Correcto	Buena mantenibilidad general del proyecto (calificación "A").
Seguridad (Security)	1 issue	Crítico	Se detectó un problema de seguridad que debe corregirse.
Fiabilidad (Reliability)	3 issues	Medio	Se identificaron algunos problemas que podrían

			afectar el correcto funcionamiento.
Duplicaciones (Duplications)	0.0%	Correcto	No hay código duplicado.
Hotspots Revisados	0.0%	Crítico	No se han revisado hotspots de seguridad, lo cual representa un riesgo.

2. DockerFile y Docker Compose en Git actions:

Dentro del workflow se puede evidenciar que se creó efectivamente el contenedor de docker utilizando el archivo dockefile y docker compose:



3. Pruebas Unitarias

❖ módulo de la calculadora

Para garantizar la correcta funcionalidad de los distintos componentes del módulo de la calculadora, se desarrollaron distintas pruebas unitarias utilizando JUnit-5. Estas pruebas permitieron verificar el comportamiento esperado de las clases encargadas del cálculo de huella de carbono.

A continuación, se hablará un poco más del propósito de cada conjunto de pruebas:

Pruebas de la clase AdapterTest: Estas pruebas verifican que la clase árboles adapter traduzco correctamente los kilómetros de CO2 emitidos a una equivalencia en árboles plantados

- Se valida un valor normal (100 kg) obteniendo *“Equivale a plantar 5 árboles.”*.
- Se prueba el caso límite de 0 kg, esperando *“Equivale a plantar 0 árboles.”*.
- Finalmente, se comprueba el manejo de valores negativos, retornando el mensaje de error *“los kilogramos de CO₂ no pueden ser menores a 0”*.

Pruebas de la clase CalculadoraHuellaTest: Aquí se evalúa la integración entre diferentes componentes del módulo

- cálculo de transporte con carro: se verifica que 100 km recorridos generen 21 kg de Co2.

- Uso de clase ArbolesAdapter donde se prueba nuevamente los mensajes de equivalencia y validación de valores negativos.
- Se prueba la creación de fuentes de energía asegurando que el cálculo total sea mayor que cero, cuando se usa “FuenteHuellaFactory”.
- Se valida que 2 fuentes combinadas generaron una suma coherente de emisiones.

Pruebas de la clase ConsolaObservadorTest: se valida el funcionamiento del patrón observador.

- Se redirige la salida estándar de la consola y se verifica que al utilizar el método actualizar () imprima correctamente un mensaje que contenga el texto “nueva huella calculada” junto con el valor calculado. Permitiendo así asegurar que las notificaciones al usuario se realiza de forma visible y adecuada.

Pruebas para EstrategiaTransporteTest y EstrategiaTest: estas dos clases verifican los distintos comportamientos del patrón Strategy aplicado en el Transporte.

- EstrategiaCarro: comprueba que se calculen 2.1 kg de CO₂ por 10 km, y que no generen emisión si la distancia es negativa.
- EstrategiaBus: valida un cálculo de 1 kg de CO₂ por 10 km.
- EstrategiaBici: confirma que no genere una emisión de CO₂.

Pruebas de FuenteHuellaFactoryTest: Evalúa la correcta creación y combinación de distintas fuentes de huella de carbono.

- Fuente de energía: Se verifica que el cálculo total coincida con la fórmula esperada (17.55 kg de CO₂).
- Fuente de dieta: Se prueba el caso de una dieta omnívora que genera 2100 kg de CO₂.
- Combinación de fuentes: Se asegura que la suma de distintas fuentes (energía y dieta) sea coherente (1005 kg de CO₂).

Estas pruebas corroboran la fiabilidad del patrón Factory utilizado para construir objetos según diferentes parámetros.

Pruebas de GrupoFuentesTest: Valida la correcta suma de emisiones en grupos de fuentes:

- Se comprueba que dos fuentes de 100 y 200 generen un total de 300 kg de CO₂.
- Se prueba un grupo vacío, que debe devolver 0.

Esto asegura la correcta agregación y composición de fuentes de emisión.

Pruebas de SujetoObservadorTest y SujetoTest: Evalúan el correcto funcionamiento del patrón Observador.

- Se comprueba que un sujeto notifique adecuadamente a uno o varios observadores.
- Se verifica que los observadores reciban exactamente el valor notificado.
- Se garantiza que el mecanismo de suscripción y notificación funcione sin errores.

Estas pruebas confirman la sincronización y comunicación entre los objetos observadores y el sujeto principal.

Pruebas de TransporteTest: Evalúan la interacción entre la clase Transporte y las distintas estrategias.

- Se valida que un transporte con estrategia de carro calcule 2.1 kg de CO₂ por 10 km.
- Se comprueba que al ingresar distancias negativas se devuelva 0.0, evitando cálculos erróneos.
- Se prueba también el caso del bus recorriendo 0 km, esperando 0.0 kg de CO₂.

Estas pruebas verifican la robustez y precisión del cálculo de emisiones en el contexto del transporte.

❖ **módulo de usuario**

Para asegurar la correcta funcionalidad de las clases pertenecientes al paquete bioprint.ModuloUsuarios, se implementaron pruebas unitarias empleando JUnit 5 y el framework Mockito para la simulación de dependencias externas.

Estas pruebas se centran en dos clases principales: Notificador (relacionada con el envío de mensajes vía Telegram) y UsuarioService (responsable de la gestión de usuarios).

A continuación, se describe el propósito y los resultados esperados de cada conjunto de pruebas.

Pruebas de la clase NotificadorTest: El objetivo de estas pruebas es verificar el correcto funcionamiento del método enviarMensaje(String mensaje) de la clase Notificador, que utiliza la API de Telegram para enviar mensajes automáticos a un chat específico.

Se aplicaron técnicas de mocking y spying mediante Mockito para evitar el envío real de mensajes y centrarse únicamente en la validación del comportamiento del método.

Pruebas desarrolladas:

- testEnviarMensajeEjecutaExecuteCorrectamente():
Comprueba que el método execute() (encargado de realizar la comunicación con Telegram) se invoque exactamente una vez cada vez que se llama a enviarMensaje().
Se utilizó un *spy* sobre el objeto Notificador para interceptar la llamada sin ejecutar la lógica real de red.
- testEnviarMensajeCreaElObjetoCorrectamente():
Verifica que el objeto SendMessage se construya correctamente antes de ser enviado.
Se captura el mensaje con un ArgumentCaptor y se comprueba que:
 - El identificador del chat sea "-4944571794".
 - El texto del mensaje corresponde exactamente con el texto proporcionado como parámetro.Esta prueba garantiza que la configuración del mensaje sea precisa antes del envío.
- testEnviarMensajeLanzaExcepcionTelegramApiException()
Evalúa la tolerancia a fallos del método frente a errores de la API de Telegram.
Se simula una excepción TelegramApiException al ejecutar execute(), y se verifica que el método enviarMensaje() no propague la excepción, asegurando así la estabilidad del sistema ante fallos externos.

Estas pruebas validan tanto el flujo normal de ejecución como el manejo de errores, garantizando la robustez y confiabilidad del proceso de notificación automática.

Pruebas de la clase UsuarioServiceTest:

Estas pruebas evalúan la lógica de negocio del servicio encargado de la gestión de usuarios, que incluye operaciones como listar, guardar y validar credenciales.

Para evitar dependencias directas con la base de datos, se utiliza un objeto `UsuarioRepository` simulado mediante Mockito, lo que permite probar el comportamiento de la capa de servicio de forma aislada.

Pruebas desarrolladas:

- `testListarDevuelveTodosLosUsuarios()`
Se simula la respuesta del repositorio con una lista de dos usuarios (juan y ana) y se comprueba que el servicio devuelva exactamente esos registros.
Se verifica además que el método `findAll()` del repositorio sea invocado una sola vez.
- `testGuardarGuardaYDevuelveUsuario()`
Comprueba que el método `guardar()` retorne correctamente el mismo usuario que se pasa como parámetro, validando la integración con el método `save()` del repositorio.
Se garantiza que la lógica de persistencia sea invocada una vez por cada solicitud de guardado.
- `testValidarUsuarioValido()`
Evalúa la validación de credenciales correctas.
Se simula que el repositorio devuelve un usuario existente cuando se buscan coincidencias por nombre y contraseña, y se espera que el método `validarUsuario()` devuelva `true`.
Esto confirma que el servicio identifica correctamente usuarios válidos.
- `testValidarUsuarioInvalido()`
Prueba el escenario contrario, donde el repositorio devuelve `null` al no encontrar coincidencias.
En este caso, `validarUsuario()` debe retornar `false`, lo que garantiza una validación segura ante credenciales incorrectas.

4. Integración con el Servicio de Mensajería

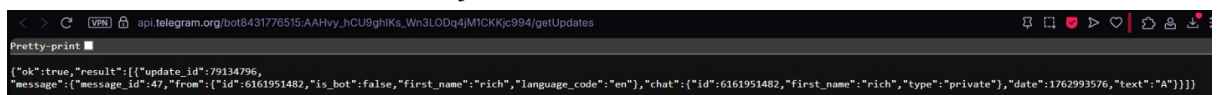
- **Configuración del bot en Telegram**

Primero, se creó un bot desde telegram usando el usuario `@BotFather`, este bot genera un token único y con este el programa puede comunicarse con la API oficial de Telegram mediante peticiones HTTP.

- Ejemplo de llamada básica:

<https://api.telegram.org/bot<token>/getUpdates>

Esto devuelve un **JSON** con los mensajes recientes enviados al bot



- **Dependencias en Maven**

En el archivo `pom.xml`, se agregaron las siguientes dependencias:

```

<dependency>
  <groupId>org.telegram</groupId>
  <artifactId>telegrambots</artifactId>
  <version>6.8.0</version>
</dependency>

<dependency>
  <groupId>org.telegram</groupId>
  <artifactId>telegrambotsextensions</artifactId>
  <version>6.8.0</version>
</dependency>

```

Estas librerías permiten manejar la conexión al servidor de Telegram y enviar/recibir mensajes desde java de forma sencilla.

- **Lógica del servicio de mensajería**

El método implementado:

```

public void enviarMensaje(String texto) {
    SendMessage mensaje = new SendMessage(chatId.toString(), texto);
    try {
        execute(mensaje);
        System.out.println("✅ Mensaje enviado a " + chatId);
    } catch (TelegramApiException e) {
        System.out.println("no se pudo enviar la notificacion");
    }
}

```

¿Qué hace?:

- SendMessage: crea un objeto mensaje con el texto que se va a enviar y el chatId del usuario destino.
- execute(mensaje): usa el método de la clase base TelegramLongPollingBot para enviar el mensaje a través de la API oficial.
- try-catch: captura errores de conexión o permisos, para evitar que el programa se detenga si ocurre un fallo.

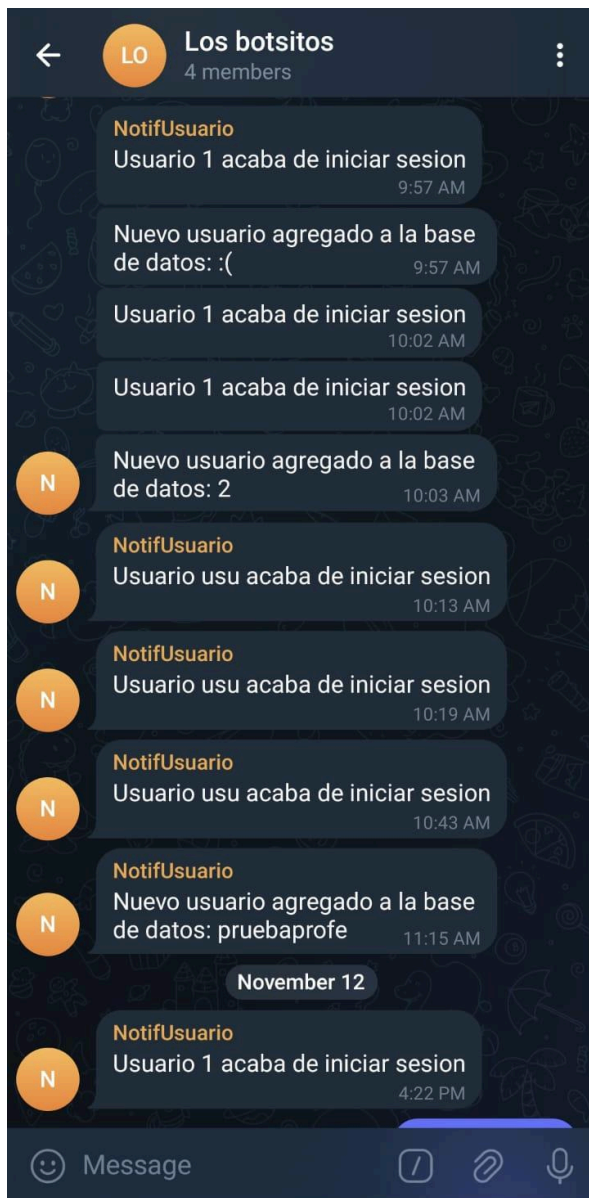
- **Obtención del chatId**

En la imagen se ve la respuesta del endpoint /getUpdates, que incluye:

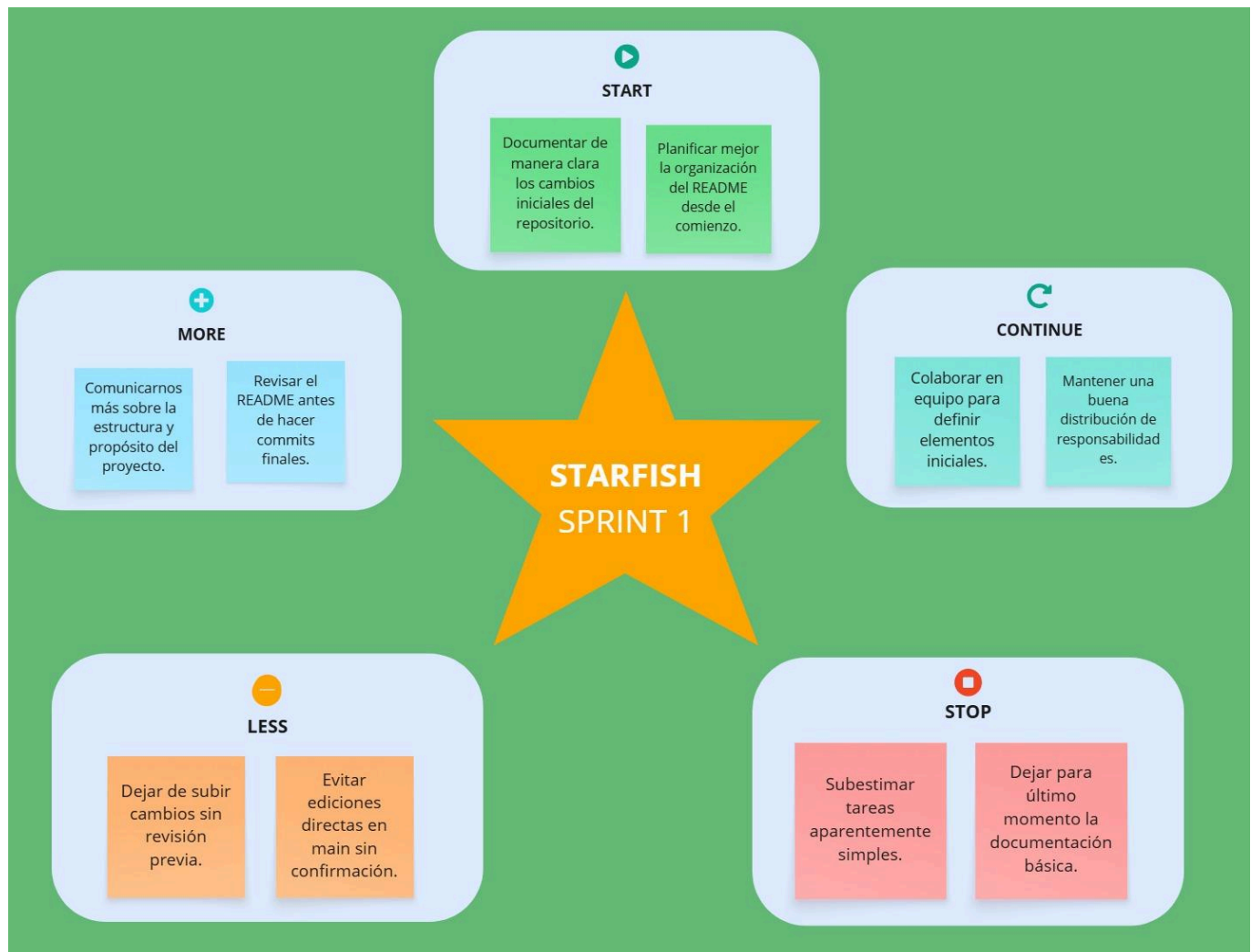
```
- "chat": {"id": 6161951482, "first_name": "rich", "type": "private"}
```

Ese chatId (6161951482) identifica al usuario con el que el bot conversa.

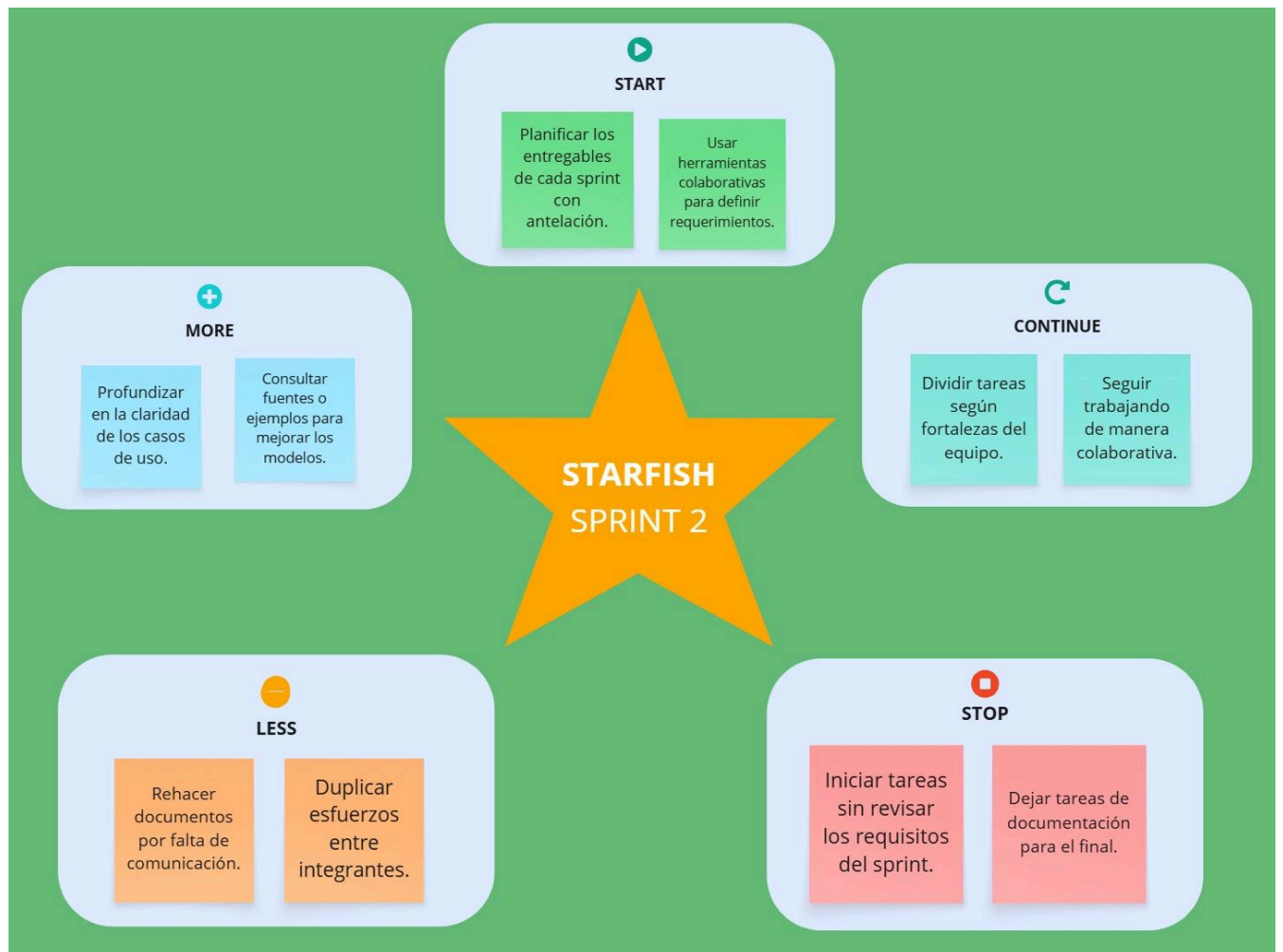
Con ese valor, el programa sabe a quién enviar el mensaje.



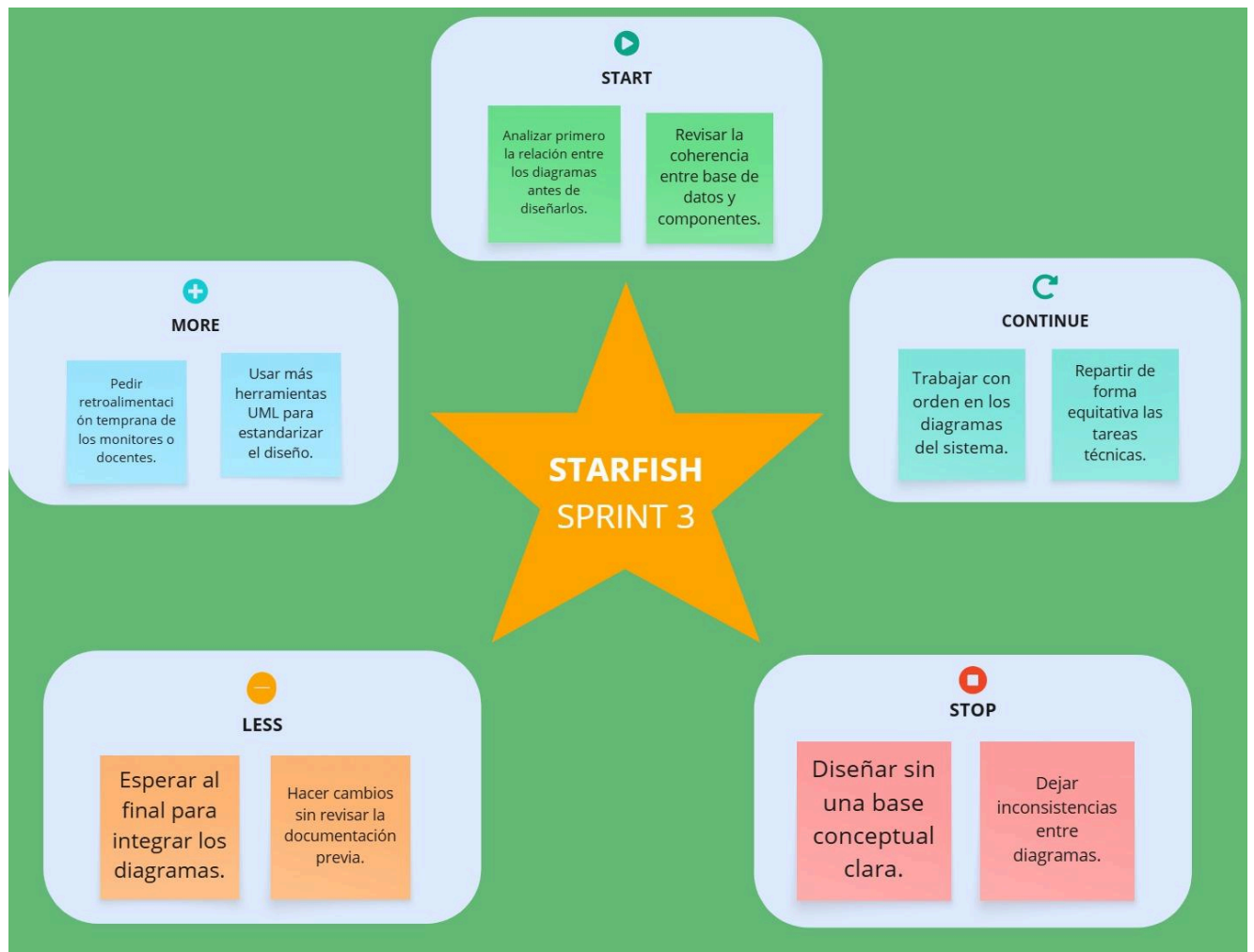
5. Enlace al Repositorio de GitHub:
https://github.com/puj-course/FIS_2530_G1
6. Retrospectiva Starfish:
 - a. Starfish por Sprint
 - i. Sprint 1:



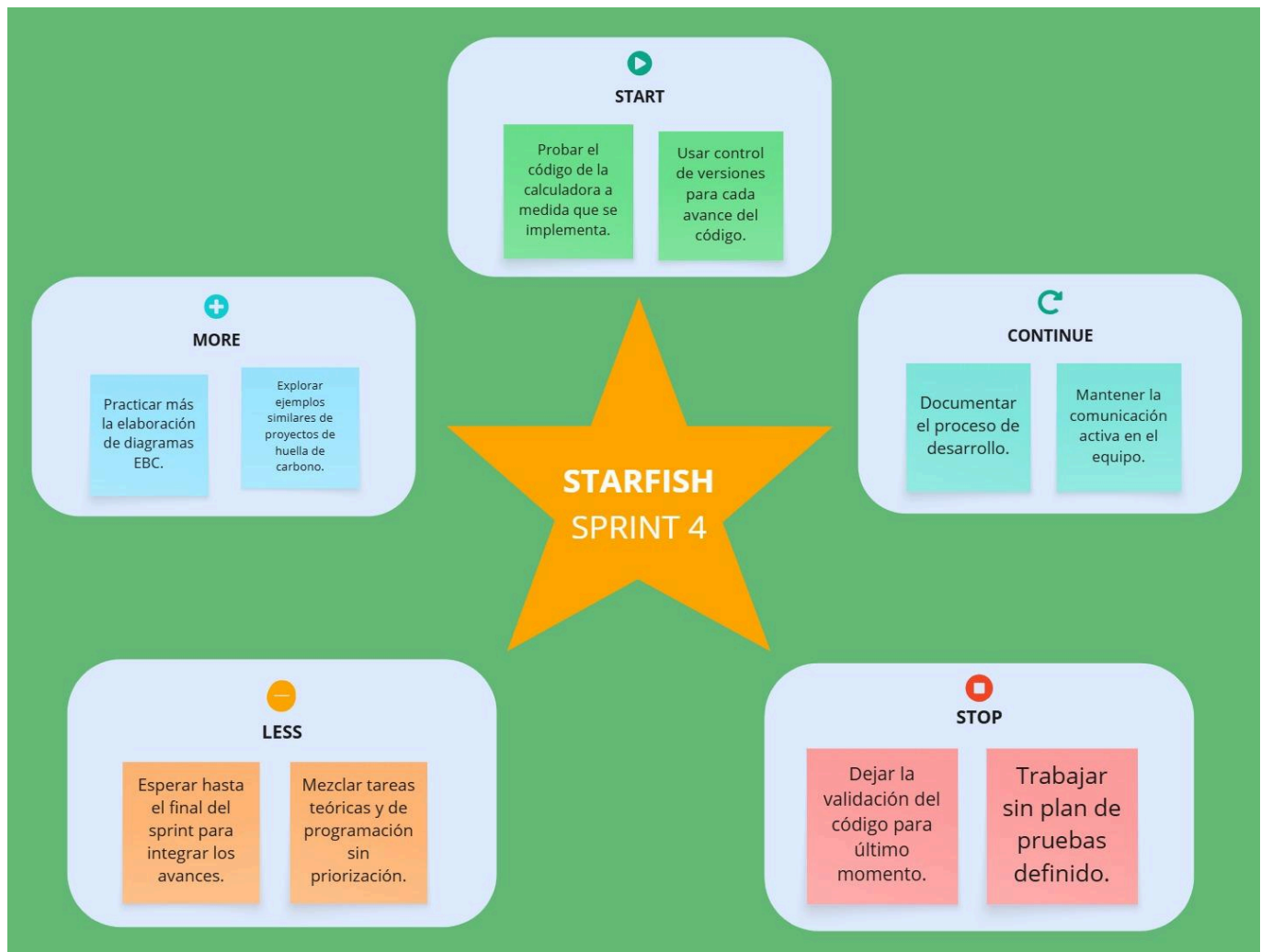
ii. Sprint 2



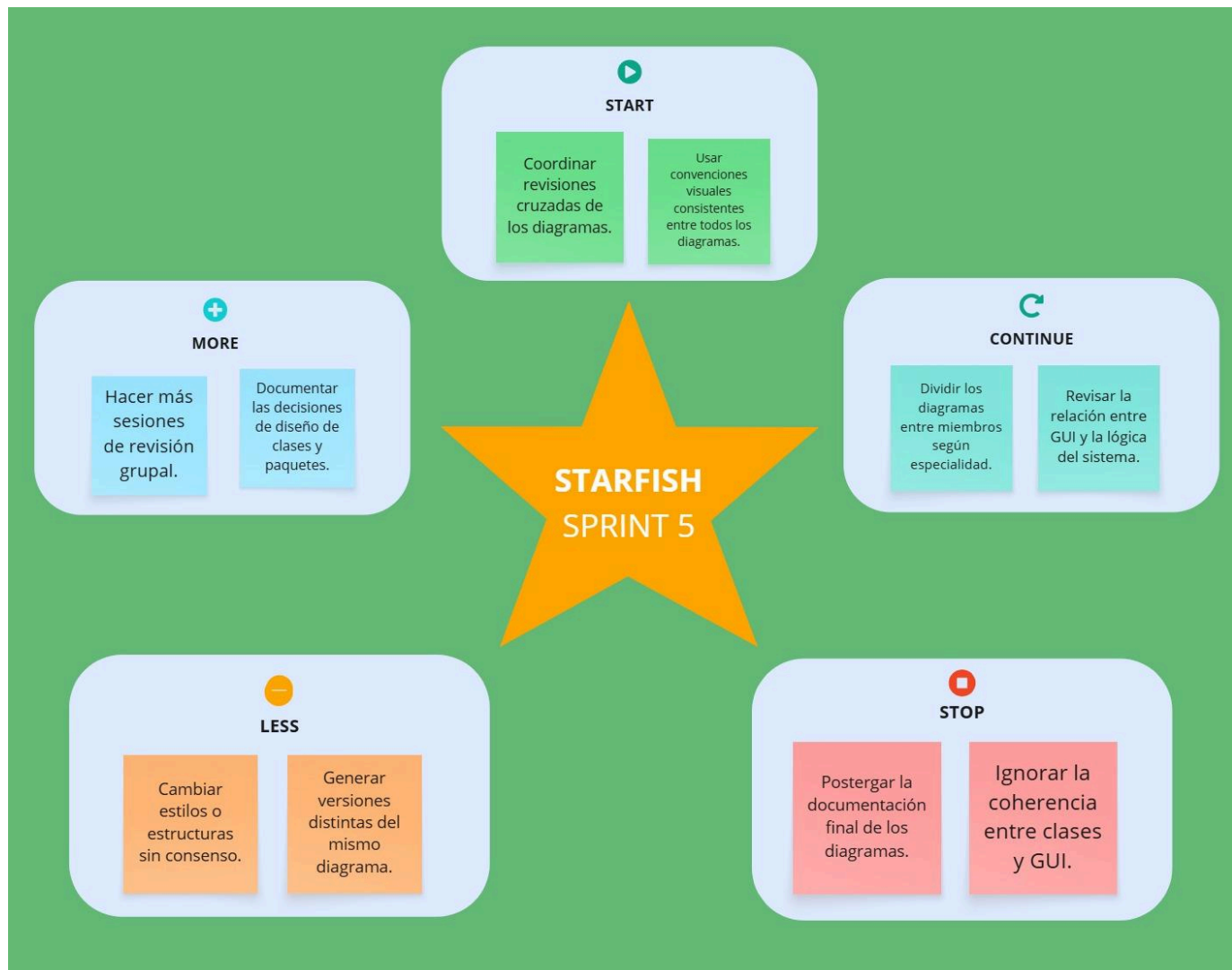
iii. Sprint 3

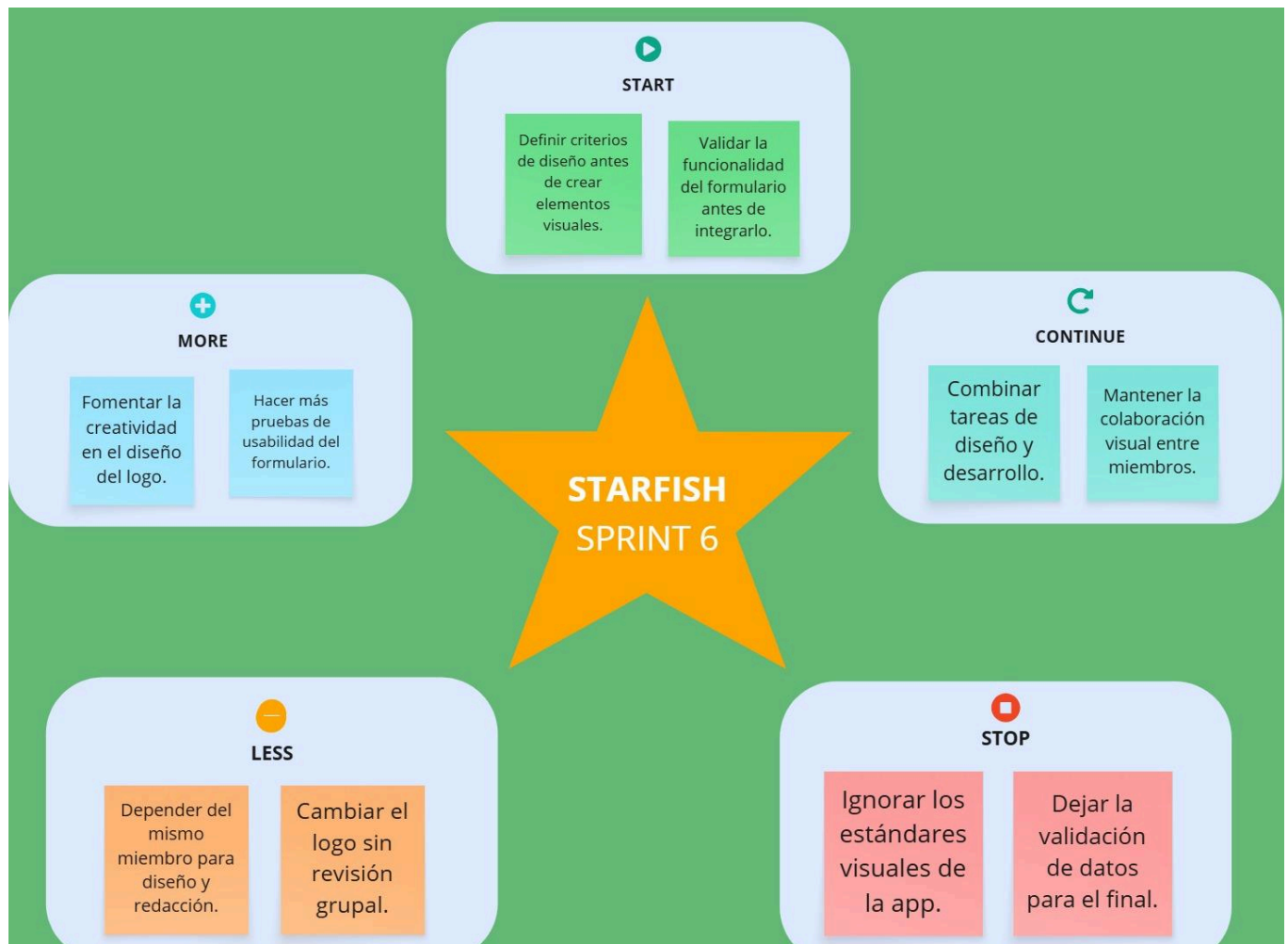


iv. Sprint 4

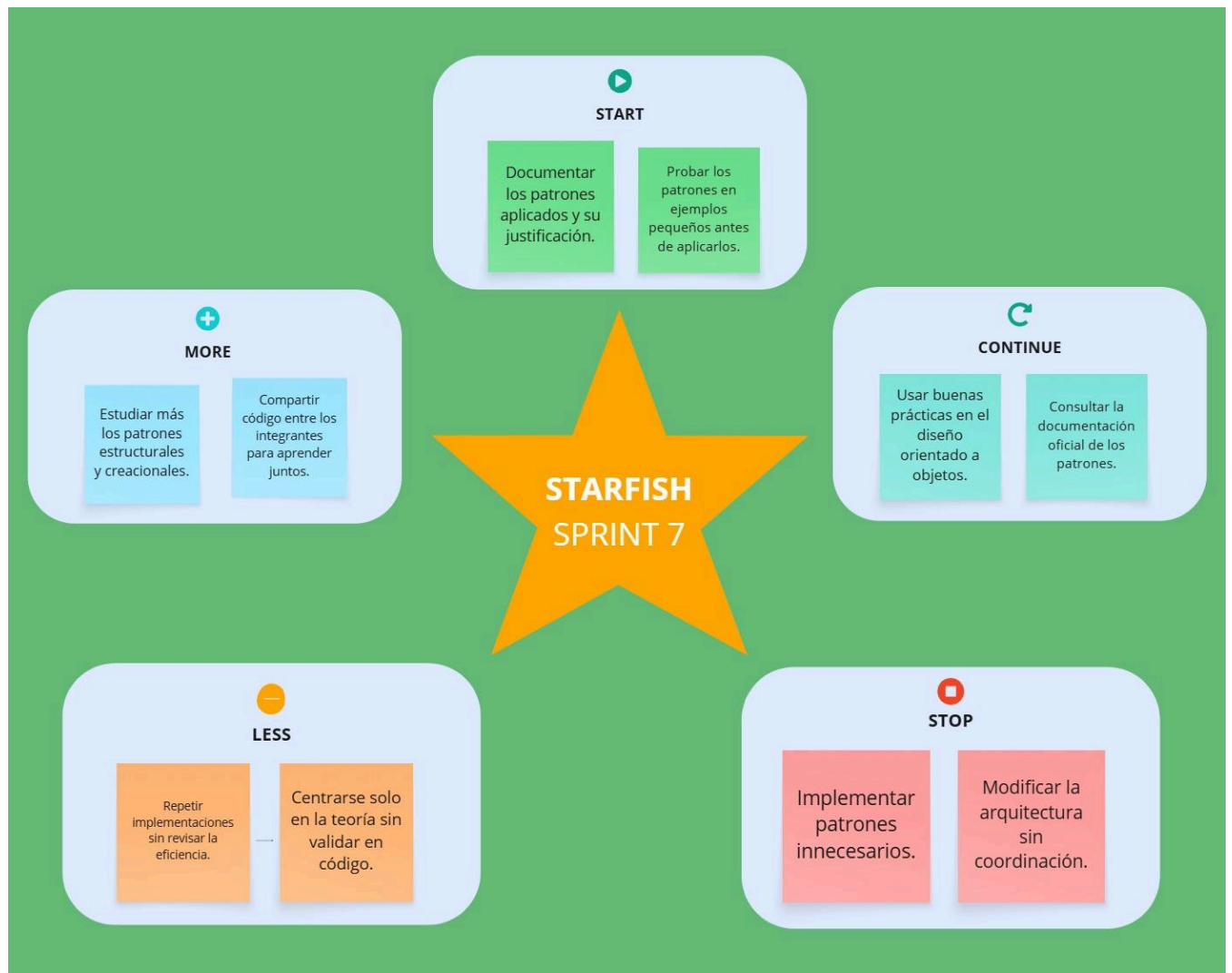


v. Sprint 5





vii. **Sprint 7**

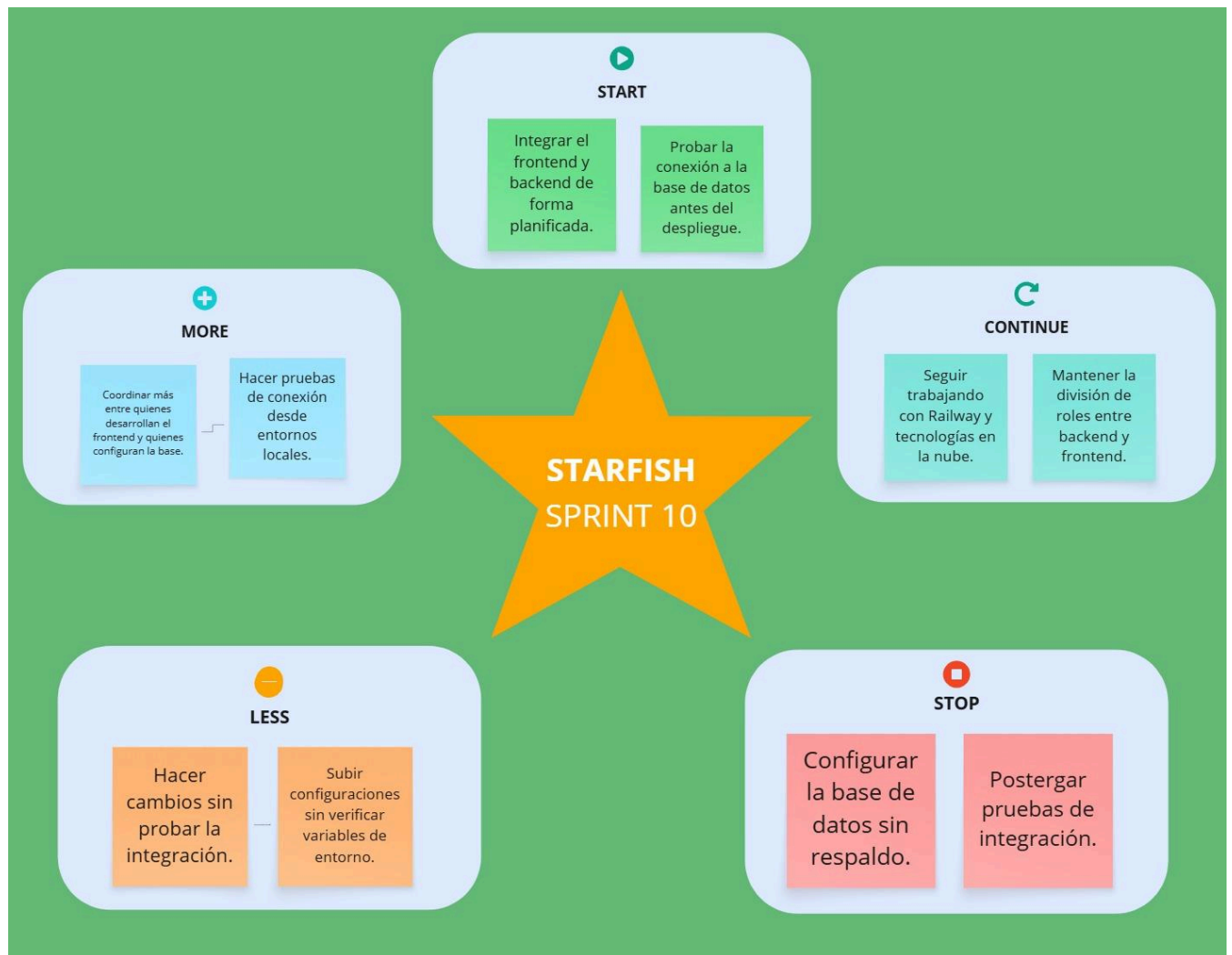


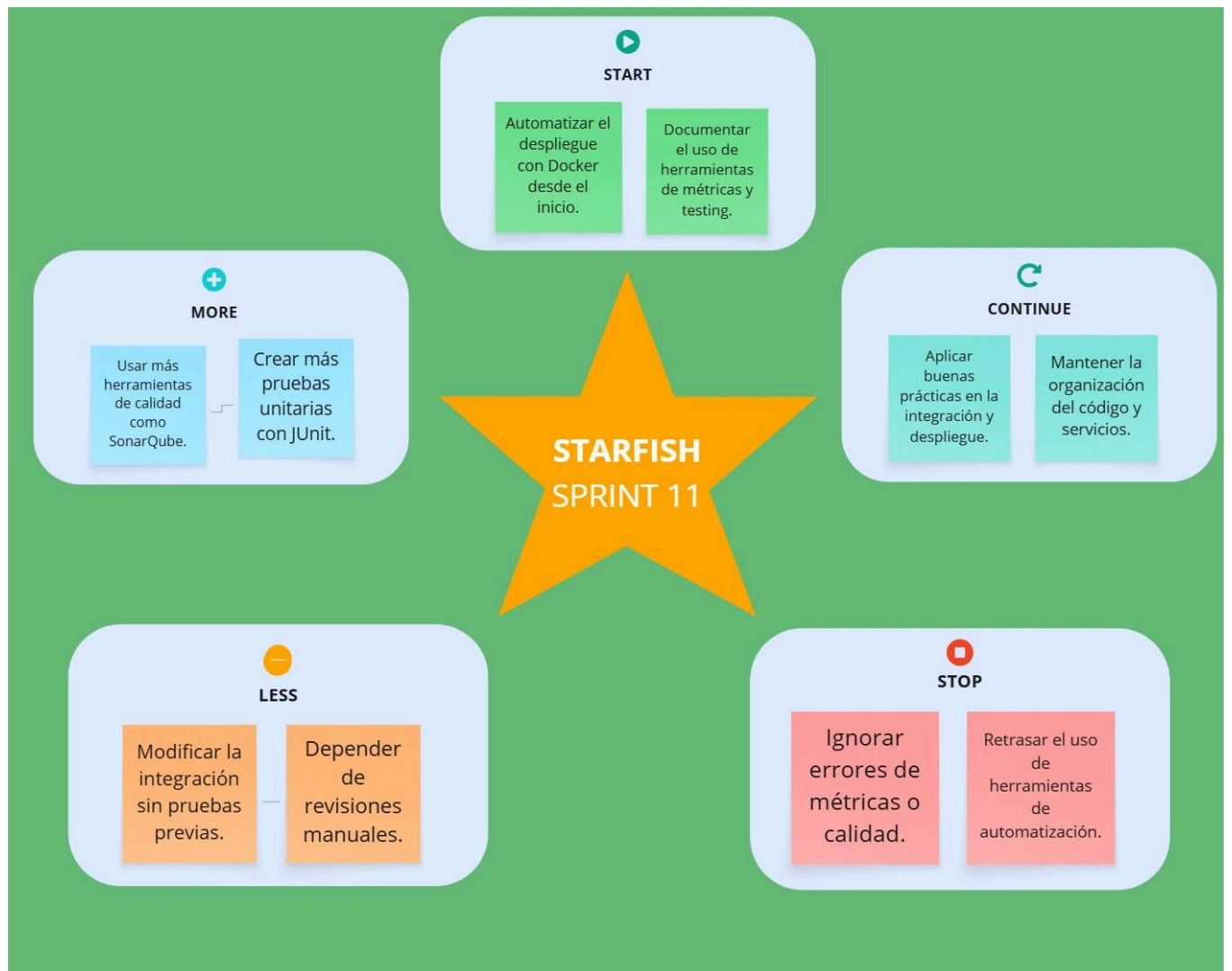


ix. Sprint 9



x. **Sprint 10**





b. Starfish General

